

Informe del Proyecto — Simulador de Gestión de Memoria

Nota: este archivo es Markdown. Para exportarlo a Word/PDF se puede usar la extensión *Markdown PDF* de VS Code, o `pandoc INFORME.md -o INFORME.docx`. Los diagramas UML están como código **PlantUML** (sección 5.1) y los placeholders `> **[IMAGEN — ...]**` están en blanco a propósito para insertar las capturas al generar el PDF.

1. Introducción

Este proyecto implementa, en Java con interfaz JavaFX, dos simuladores de administración de memoria:

- **Traducción de direcciones virtuales a físicas** mediante paginación de un solo nivel (tabla de páginas).
- **Reemplazo de páginas** con las tres políticas clásicas **FIFO**, **LRU** y **OPT**, ejecutadas sobre la misma cadena de referencias para poder compararlas.

Los casos de prueba se cargan desde archivos `.txt` en la carpeta `archivosTXT/`.

2. Objetivos

- Implementar la traducción de direcciones virtuales a físicas.
- Implementar y comparar los algoritmos de reemplazo **FIFO**, **LRU** y **OPT** sobre la misma cadena.
- Mostrar resultados paso a paso (estado de marcos, fallos y aciertos).
- Permitir la carga de datos desde archivos TXT.
- Proveer una interfaz JavaFX clara, sin librerías externas.

3. Arquitectura del proyecto

Se siguió una separación por capas inspirada en MVC, agrupada en paquetes dentro de `src/`:

Paquete	Responsabilidad	Clases
(raíz <code>src/</code>)	Punto de entrada	<code>Main</code>
<code>dominio</code>	Entidades puras del problema	<code>Bloque</code> , <code>Casilla</code> , <code>RAM</code> , <code>AlmacenamientoVirtual</code> , <code>MapaPaginas</code> , <code>ReporteTraduccion</code> , <code>ReporteAlgoritmo</code>
<code>logica</code>	Algoritmos de traducción y reemplazo	<code>Traductor</code> , <code>PoliticaFIFO</code> , <code>PoliticaLRU</code> , <code>PoliticaOPT</code>
<code>persistencia</code>	Lectura de archivos TXT	<code>LectorTXT</code>
<code>herramientas</code>	Conversión y validación de datos	<code>Parseador</code>

Paquete	Responsabilidad	Clases
coordinador	Une vista, lógica y persistencia	Coordinador
interfaz	UI con JavaFX	PanelPrincipal

Reglas de dependencia: `interfaz` solo conoce a `coordinador`; `coordinador` conoce a `logica`, `persistencia`, `herramientas` y `dominio`; `logica` y `persistencia` solo dependen de `dominio` y `herramientas`; `dominio` no depende de ningún otro paquete del proyecto.

4. Algoritmos implementados

4.1 Traducción de direcciones (paginación simple)

Una dirección virtual `dv` se descompone en `pagina = dv / tamPag` y `offset = dv % tamPag`. La tabla de páginas (`MapaPaginas`) indica en qué casilla de RAM está cargada cada página. Si la página está cargada, la dirección física se reconstruye como `casilla * tamPag + offset`; si no, se reporta **fallo de página**. Antes de calcular se valida que las memorias sean múltiplos del tamaño de página y que la dirección esté dentro del rango. Vive en `logica/Traductor.java` (método `resolver`). Para esta simulación el mapa se carga en identidad (`i → i`) en las primeras casillas.

4.2 FIFO

La víctima es la página que **lleva más tiempo cargada**. Se modela con un `ArrayDeque`: cuando entra una referencia y no hay casillas libres, sale la del frente. Complejidad $O(n)$ sobre la cadena. FIFO puede sufrir la anomalía de Belady (más marcos $\Rightarrow$ más fallos).

4.3 LRU

La víctima es la página cuyo **último uso** ocurrió hace más tiempo. Se recorre la cadena hacia atrás para encontrar la última aparición de cada marco y se victimiza la más lejana. LRU es política de tipo *stack*, por lo que **no** sufre la anomalía de Belady.

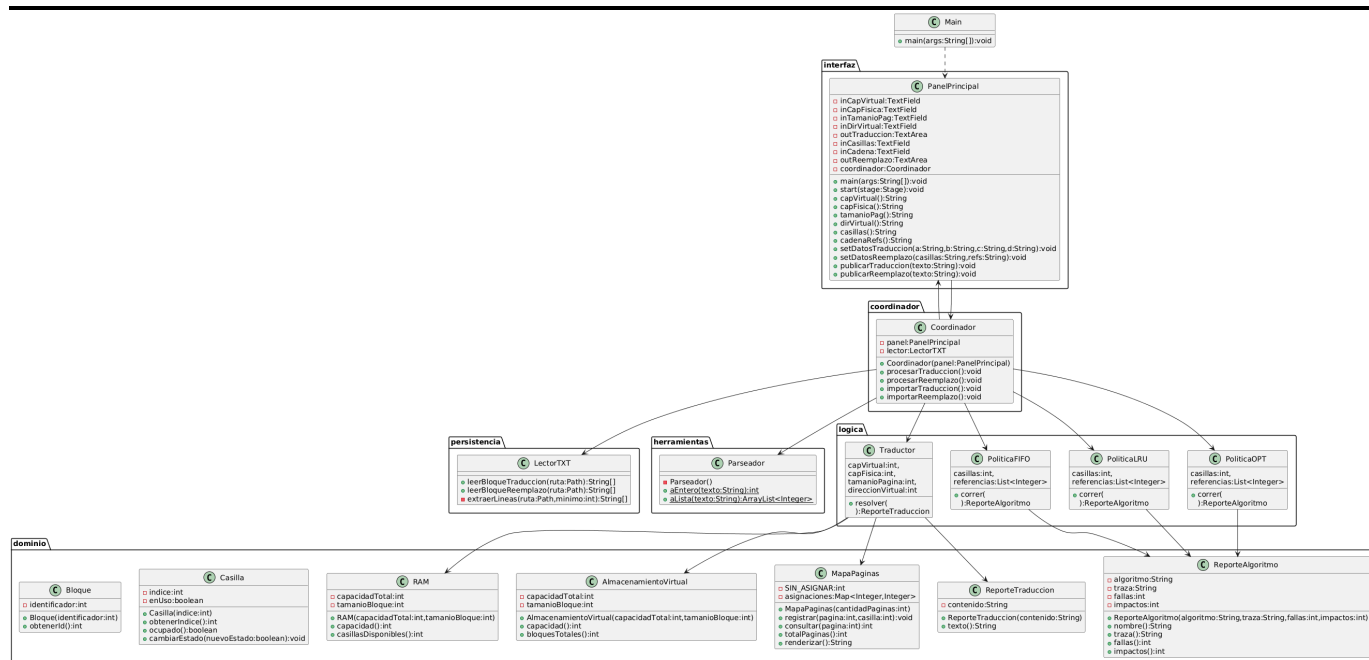
4.4 OPT (Óptimo de Belady)

La víctima es la página que **se usará más lejos en el futuro** (o que ya no se vuelve a usar). Requiere conocer toda la cadena, así que no es implementable en sistemas reales, pero sirve como **techo teórico** del mínimo de fallos.

4.5 Comparación

Política	Criterio de víctima	Mira...	Anomalía de Belady	Uso real
FIFO	Más antigua en RAM	Pasado de cargas	Sí	Raro
LRU	Menos usada reciente	Pasado de accesos	No	Común
OPT	Más lejana en futuro	El futuro	No	Solo teórico

5. Diagramas UML



6. Casos de prueba y archivos de ejemplo

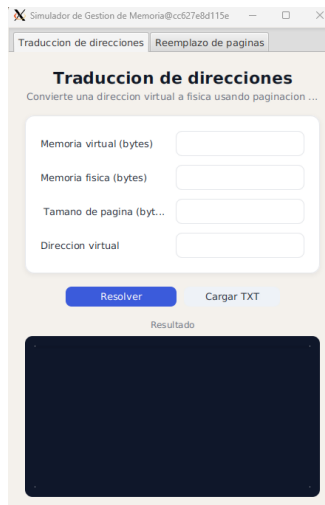
En la carpeta **archivosTXT/** se incluyen los siguientes casos, todos verificables cargándolos con el botón *Cargar TXT* de la app.

#	Archivo .txt	Tabla	Resultado esperado
1	traduccion_exitosa.txt	Traducción	dv=300 → df=300 (página 2 en RAM)
2	traduccion_fallo_pagina.txt	Traducción	dv=1800 → FALLO (página 14 no cargada)
3	reemplazo_cadena_clasica.txt	Reemplazo	FIFO = 12/5 , LRU = 10/7 , OPT = 8/9 (fallas/impactos)
4	reemplazo_4marcos.txt	Reemplazo	FIFO = 10/2 , LRU = 8/4 , OPT = 6/6
5	reemplazo_belady.txt	Reemplazo	FIFO = 9/3 , LRU = 9/3 , OPT = 7/5 (3 marcos; con 4 marcos FIFO sube a 10: anomalía de Belady)
6	reemplazo_corto.txt	Reemplazo	FIFO = 6/4 , LRU = 7/3 , OPT = 6/4

Los conteos **fallas/impactos** se reimprimen tal como los produce la app (formato **Fallas: N** | **Impactos: M**). En la cadena clásica (caso 3) se confirma la jerarquía teórica **OPT < LRU < FIFO**.

7. Capturas de ejecución

- **Ventana principal al iniciar:** las dos pestañas y los campos vacíos.



- **Traducción exitosa:** pestaña *Traducción* con 2048 / 512 / 128 / 300 y el TextArea mostrando **Dir. fisica = 300**.



Resultado:

=== TRADUCCION DE DIRECCIONES ===

Cap. virtual : 2048

Cap. RAM : 512

Tamano pag. : 128

Bloques virt. : 16

Casillas RAM : 4

--- MAPA DE PAGINAS ---

Pag 0 -> Casilla 0

Pag 1 -> Casilla 1

Pag 2 -> Casilla 2

Pag 3 -> Casilla 3

Pag 4 -> fuera de RAM

Pag 5 -> fuera de RAM

Pag 6 -> fuera de RAM

Pag 7 -> fuera de RAM
 Pag 8 -> fuera de RAM
 Pag 9 -> fuera de RAM
 Pag 10 -> fuera de RAM
 Pag 11 -> fuera de RAM
 Pag 12 -> fuera de RAM
 Pag 13 -> fuera de RAM
 Pag 14 -> fuera de RAM
 Pag 15 -> fuera de RAM

--- PROCEDIMIENTO ---

Dir. virtual ingresada: 300

Pagina = $300 / 128 = 2$

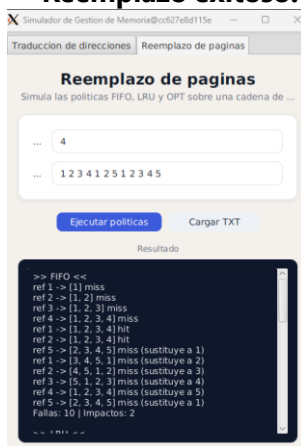
Offset = $300 \% 128 = 44$

La pagina 2 esta en la casilla 2.

Dir. fisica = $2 * 128 + 44$

Dir. fisica = 300

• **Reemplazo exitoso:** 4 • 1 2 3 4 1 2 5 1 2 3 4 5.



Resultado:

>> FIFO <<

ref 1 -> [1] miss

ref 2 -> [1, 2] miss

ref 3 -> [1, 2, 3] miss

ref 4 -> [1, 2, 3, 4] miss

ref 1 -> [1, 2, 3, 4] hit

ref 2 -> [1, 2, 3, 4] hit

ref 5 -> [2, 3, 4, 5] miss (sustituye a 1)

ref 1 -> [3, 4, 5, 1] miss (sustituye a 2)

ref 2 -> [4, 5, 1, 2] miss (sustituye a 3)

ref 3 -> [5, 1, 2, 3] miss (sustituye a 4)

ref 4 -> [1, 2, 3, 4] miss (sustituye a 5)

ref 5 -> [2, 3, 4, 5] miss (sustituye a 1)

Fallas: 10 | Impactos: 2

>> LRU <<

ref 1 -> [1] miss

ref 2 -> [1, 2] miss

ref 3 -> [1, 2, 3] miss

ref 4 -> [1, 2, 3, 4] miss

ref 1 -> [1, 2, 3, 4] hit

```
ref 2 -> [1, 2, 3, 4] hit
ref 5 -> [1, 5, 3, 4] miss (sustituye a 2)
ref 1 -> [1, 5, 3, 4] hit
ref 2 -> [2, 5, 3, 4] miss (sustituye a 1)
ref 3 -> [2, 5, 3, 4] hit
ref 4 -> [2, 5, 3, 4] hit
ref 5 -> [2, 5, 3, 4] hit
Fallas: 6 | Impactos: 6
```

```
>> OPT <<
ref 1 -> [1] miss
ref 2 -> [1, 2] miss
ref 3 -> [1, 2, 3] miss
ref 4 -> [1, 2, 3, 4] miss
ref 1 -> [1, 2, 3, 4] hit
ref 2 -> [1, 2, 3, 4] hit
ref 5 -> [1, 2, 3, 5] miss (sustituye a 4)
ref 1 -> [1, 2, 3, 5] hit
ref 2 -> [1, 2, 3, 5] hit
ref 3 -> [1, 2, 3, 5] hit
ref 4 -> [4, 2, 3, 5] miss (sustituye a 1)
ref 5 -> [4, 2, 3, 5] hit
Fallas: 6 | Impactos: 6
```

```
=== RESUMEN COMPARATIVO ===
FIFO -> fallas: 10 | impactos: 2
LRU -> fallas: 6 | impactos: 6
OPT -> fallas: 6 | impactos: 6
```

8. Análisis y conclusión del proyecto

Con este trabajo aprendí a usar Docker para empaquetar y correr la aplicación de forma reproducible (Dockerfile y docker-compose.yml en la raíz del proyecto), y también a manejar .gitignore para no versionar artefactos como out/ ni el SDK de JavaFX. En la parte de Java reforcé el trabajo con clases por capas, Genéricos, Map, Deque, lectura de archivos con java.nio.file y la separación entre la lógica y la interfaz JavaFX.

La inteligencia artificial fue utilizada como herramienta de apoyo durante el desarrollo de este proyecto. Se empleó para la implementación y análisis de código, el aprendizaje de tecnologías como Docker y procesos de compilación, así como para mejorar la presentación y elaboración de parte de la documentación. Todo el contenido generado fue revisado y validado manualmente antes de su uso, sirviendo la IA únicamente como una ayuda complementaria durante el desarrollo.

GitHub →

Youtube →